MAS1004 Data & AI, 2026 Fall

Assignment 1: Long report

Name: Isabel Mendoza Garcia Student ID: 20231987

Problem 1. Categories and images

Why these categories

My three categories are the soles of three shoe models: `converse_diamond_sole_shape`, `nike_air_force_circula` and `vans_honeycomb_sole_shape`. Each name is a tread pattern: the diamond grid of a Converse, the clean lines and circles of a Nike Air Force 1 and the honeycomb of a Vans.

I know shoe soles can be a random category. I decided to do this because I am a dancer. For urban dances, although some don't really pay much attention, we should wear certain shoes with certain soles depending on the style. For example, hip hop needs shoes that will have grip and give steadiness to your ankles, and makes it easier to be grounded and avoid injuries. For house you might need lighter shoes to move swiftly and faster, for turns, soles with circular shapes might help, etc. I thought it would be fun to do an image classifier for soles, and find similar soles to certain brand in case one doesn't have the chance to buy the exact shoe brand or model recommended by many, but can be at least similar.

What I expected the model to use

I wrote this guess again after the first training runs, because the guess I wrote at the start of the assignment was not saved. It is a rewritten guess, not the original.

I guess it will use shape, color, and texture. Shape for the type of lines or basic shape of the sole, texture for the grid the sole has, and color as it might help identify shadows and highlights and determine a more exact shape.

Images per category

I collected the images twice. My first collection (150 images per category, 450 in all) gave only 204 usable images after cleaning (62, 67 and 75), so I collected again with more search phrases and replaced it. The first-round data is kept in `data/backup_before_recollect`. The second collection used several search phrases per category (150 images each), and I merged the phrases into the three categories, skipping exact duplicates:

- Converse: "converse shoe sole", "chuck taylor all star sole", then, because too few usable images were left, "converse chuck taylor all star outsole", "converse all star sole bottom view", "converse chuck taylor sole pattern" and "converse diamond tread sole".
- Nike: "nike air force sole", "nike air force shoe sole", "nike air force outsole" and "nike air force outsole circular shape".
- Vans: "vans outsole shape", "vans waffle sole" and "vans old skool honeycomb outsole".

category	downloaded images
converse_diamond_sole_shape	700
nike_air_force_circular_sole_shape	452
vans_honeycomb_sole_shape	427
total	1,579

Run 1 printed the same counts (`outputs/run_output.txt`). The merge printed 289, 452 and 427 images after the first nine phrases; 411 Converse images were added afterwards as files 0290 to 0700.

Problem 2. Preparing an image



the image as downloaded



what `prepare_image` made of it

The picture is a Nike sole from my first collection round (the folders were rebuilt later, so its file number no longer exists). It is 1600 x 566 pixels. `prepare_image` resizes the shorter side to 256 (723 x 256) and cuts out the middle 224 x 224 square. 499 pixels of the long side are thrown away and only 31.0% of the long side is kept (printed in `results/cleaning/prepare_numbers.txt`).

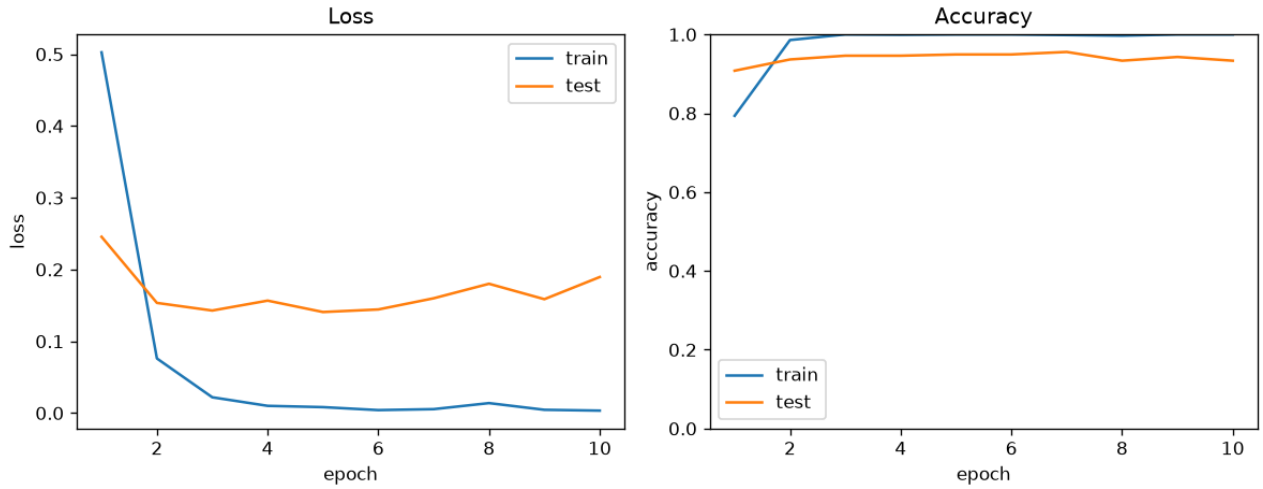
It cuts off the beginning and end of the sole, so we can see the middle, the circular part of the sole and part of the logo. The pattern was cropped but it is still something you can identify clearly.

Problem 3. First training runs

All four runs used all 1,579 downloaded images, before any cleaning: 1,264 for training and 315 for testing.

tag	start	trained	epochs	lr	trainable parameters	train acc.	test acc.
run	ImageNet	all layers	10	0.0001	11,178,051	100.0%	93.3%
scratch	random	all layers	10	0.0001	11,178,051	99.9%	65.1%
frozen	ImageNet	last layer	10	0.001	1,539	87.9%	85.7%
long	ImageNet	all layers	20	0.0003	11,178,051	100.0%	94.0%

The fourth run (`long`) is the setting of my own: 20 epochs and a higher learning rate (I took the recommended setting).

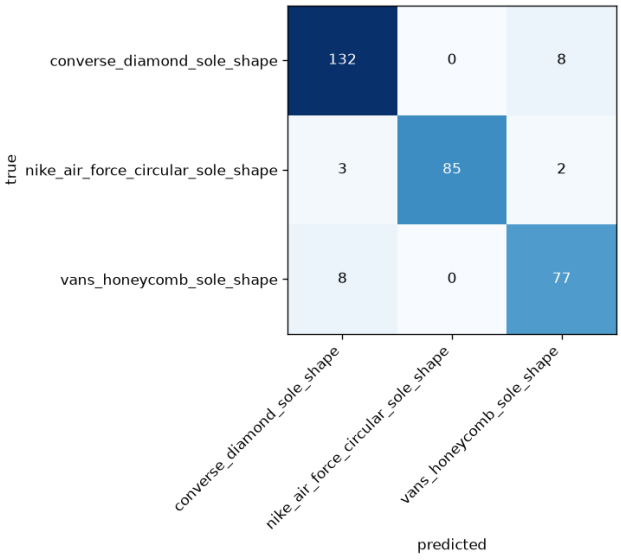


The loss

The training loss goes from 0.50 to 0.003, so it keeps falling and gets very small. The test loss reaches its lowest value, 0.14, around epochs 3 to 5, then drifts up to 0.19. Numbers: training loss 0.5025 in epoch 1, 0.0761 in epoch 2 and 0.0034 in epoch 10; test loss lowest at 0.1408 in epoch 5 and 0.1895 in epoch 10 (`outputs/run_output.txt`).

Training against test accuracy

Train accuracy is 100.0% and test accuracy is 93.3% (run), so train is 6.7 points higher. The model has memorised the training images almost perfectly but is less sure on images it has not seen. A gap this size means some overfitting, but the test accuracy is still high.



The confusion matrix

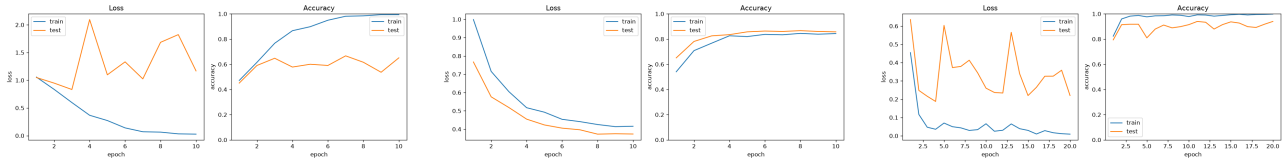
The most confused classes are Converse and Vans. I am not surprised. I think those two are the most similar in terms of shape, at a certain angle or with a lot of zoom, it might get confused. Compared to the Nike model, which is very different, and also the color of the sole is mostly white instead of the brownish tone the other 2 have. Numbers from the run confusion matrix (315 test images): 8 Converse images were called Vans and 8 Vans images were called Converse. Nike was the most reliable, with 85 of 90 correct (3 called Converse, 2 called Vans).

ImageNet start against random start

Starting from ImageNet gave 93.3% test accuracy and starting from random numbers gave 65.1%, a gap of 28.2 points. The scratch run also jumped around (test accuracy between 54% and 67% across epochs). A reasonable explanation is that the ImageNet weights already know how to see edges, textures and shapes, so with 1,264 training images the model only has to adjust them, while a random start has to learn everything from the same few images.

Training only the last layer

Training only the last layer (1,539 parameters, about 0.014% of 11,178,051) gave 85.7% test accuracy, against 93.3% for training everything, so it gets within 7.6 points. Its train accuracy was also lower (87.9% against 100%): it did not overfit, but it could not fit the data as closely.



Output of the runs

```
reading data/raw
1579 images, 3 classes, read in 28 s
  converse_diamond_sole_shape 700
  nike_air_force_circular_sole_shape 452
  vans_honeycomb_sole_shape 427
training on 1264, testing on 315
model has 11,178,051 parameters, 11,178,051 of them will be trained
epoch 1/10  train loss 0.5025  train acc 0.794  test loss 0.2455  test acc 0.908
epoch 2/10  train loss 0.0761  train acc 0.986  test loss 0.1535  test acc 0.937
epoch 3/10  train loss 0.0220  train acc 1.000  test loss 0.1428  test acc 0.946
epoch 4/10  train loss 0.0101  train acc 0.999  test loss 0.1566  test acc 0.946
epoch 5/10  train loss 0.0083  train acc 1.000  test loss 0.1408  test acc 0.949
epoch 6/10  train loss 0.0041  train acc 1.000  test loss 0.1443  test acc 0.949
epoch 7/10  train loss 0.0054  train acc 0.998  test loss 0.1599  test acc 0.956
epoch 8/10  train loss 0.0139  train acc 0.997  test loss 0.1801  test acc 0.933
epoch 9/10  train loss 0.0045  train acc 1.000  test loss 0.1586  test acc 0.943
epoch 10/10  train loss 0.0034  train acc 1.000  test loss 0.1895  test acc 0.933

trained in 6.6 minutes
train accuracy 100.0%
test  accuracy 93.3%
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\run_curves.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\run_confusion.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment 1\mas1004-assignment1\results\run_worst.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment 1\mas1004-assignment1\results\run_model.pt

One line for your experiment table:
| run | ImageNet | all layers | 10 | 0.0001 | 11,178,051 | 100.0% | 93.3% |

To put this model on your web page: python src/export_web.py --tag run
```

```
reading data/raw
1579 images, 3 classes, read in 26 s
  converse_diamond_sole_shape 700
  nike_air_force_circular_sole_shape 452
  vans_honeycomb_sole_shape 427
training on 1264, testing on 315
model has 11,178,051 parameters, 11,178,051 of them will be trained
epoch 1/10  train loss 1.0569  train acc 0.472  test loss 1.0493  test acc 0.451
epoch 2/10  train loss 0.8351  train acc 0.616  test loss 0.9483  test acc 0.590
epoch 3/10  train loss 0.5977  train acc 0.767  test loss 0.8330  test acc 0.648
epoch 4/10  train loss 0.3682  train acc 0.866  test loss 2.0946  test acc 0.578
epoch 5/10  train loss 0.2731  train acc 0.898  test loss 1.0982  test acc 0.600
epoch 6/10  train loss 0.1406  train acc 0.949  test loss 1.3320  test acc 0.590
epoch 7/10  train loss 0.0707  train acc 0.981  test loss 1.0248  test acc 0.667
epoch 8/10  train loss 0.0641  train acc 0.984  test loss 1.6842  test acc 0.616
epoch 9/10  train loss 0.0328  train acc 0.994  test loss 1.8232  test acc 0.537
epoch 10/10  train loss 0.0278  train acc 0.994  test loss 1.1684  test acc 0.651

trained in 6.6 minutes
train accuracy 99.9%
test  accuracy 65.1%
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\scratch_curves.png
```

```
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\scratch_confusion.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\scratch_worst.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\scratch_model.pt

One line for your experiment table:
| scratch | random | all layers | 10 | 0.0001 | 11,178,051 | 99.9% | 65.1% |

To put this model on your web page: python src/export_web.py --tag scratch
```

```
reading data/raw
1579 images, 3 classes, read in 26 s
  converse_diamond_sole_shape 700
  nike_air_force_circular_sole_shape 452
  vans_honeycomb_sole_shape 427
training on 1264, testing on 315
model has 11,178,051 parameters, 1,539 of them will be trained
epoch 1/10  train loss 0.9988  train acc 0.540  test loss 0.7670  test acc 0.651
epoch 2/10  train loss 0.7157  train acc 0.708  test loss 0.5766  test acc 0.781
epoch 3/10  train loss 0.6049  train acc 0.767  test loss 0.5181  test acc 0.825
epoch 4/10  train loss 0.5172  train acc 0.826  test loss 0.4547  test acc 0.835
epoch 5/10  train loss 0.4927  train acc 0.820  test loss 0.4230  test acc 0.857
epoch 6/10  train loss 0.4537  train acc 0.837  test loss 0.4055  test acc 0.863
epoch 7/10  train loss 0.4412  train acc 0.834  test loss 0.3966  test acc 0.860
epoch 8/10  train loss 0.4253  train acc 0.845  test loss 0.3727  test acc 0.867
epoch 9/10  train loss 0.4137  train acc 0.839  test loss 0.3749  test acc 0.860
epoch 10/10  train loss 0.4158  train acc 0.843  test loss 0.3734  test acc 0.857

trained in 2.4 minutes
train accuracy 87.9%
test  accuracy 85.7%
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\frozen_curves.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\frozen_confusion.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\frozen_worst.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\frozen_model.pt

One line for your experiment table:
| frozen | ImageNet | last layer | 10 | 0.001 | 1,539 | 87.9% | 85.7% |

To put this model on your web page: python src/export_web.py --tag frozen
```

```
reading data/raw
1579 images, 3 classes, read in 38 s
  converse_diamond_sole_shape 700
  nike_air_force_circular_sole_shape 452
  vans_honeycomb_sole_shape 427
training on 1264, testing on 315
model has 11,178,051 parameters, 11,178,051 of them will be trained
epoch 1/20  train loss 0.4545  train acc 0.823  test loss 0.6353  test acc 0.794
epoch 2/20  train loss 0.1171  train acc 0.960  test loss 0.2483  test acc 0.914
epoch 3/20  train loss 0.0472  train acc 0.982  test loss 0.2163  test acc 0.917
epoch 4/20  train loss 0.0358  train acc 0.987  test loss 0.1878  test acc 0.917
epoch 5/20  train loss 0.0698  train acc 0.976  test loss 0.6035  test acc 0.810
epoch 6/20  train loss 0.0499  train acc 0.984  test loss 0.3730  test acc 0.879
epoch 7/20  train loss 0.0436  train acc 0.985  test loss 0.3792  test acc 0.911
epoch 8/20  train loss 0.0296  train acc 0.991  test loss 0.4132  test acc 0.889
epoch 9/20  train loss 0.0337  train acc 0.989  test loss 0.3444  test acc 0.898
epoch 10/20  train loss 0.0660  train acc 0.979  test loss 0.2604  test acc 0.914
```

```
epoch 11/20  train loss 0.0251  train acc 0.992  test loss 0.2368  test acc 0.940
epoch 12/20  train loss 0.0306  train acc 0.991  test loss 0.2340  test acc 0.933
epoch 13/20  train loss 0.0650  train acc 0.982  test loss 0.5656  test acc 0.879
epoch 14/20  train loss 0.0391  train acc 0.987  test loss 0.3396  test acc 0.914
epoch 15/20  train loss 0.0297  train acc 0.993  test loss 0.2201  test acc 0.937
epoch 16/20  train loss 0.0100  train acc 0.997  test loss 0.2651  test acc 0.927
epoch 17/20  train loss 0.0284  train acc 0.991  test loss 0.3257  test acc 0.898
epoch 18/20  train loss 0.0169  train acc 0.995  test loss 0.3260  test acc 0.892
epoch 19/20  train loss 0.0116  train acc 0.996  test loss 0.3585  test acc 0.917
epoch 20/20  train loss 0.0092  train acc 0.998  test loss 0.2204  test acc 0.940

trained in 16.9 minutes
train accuracy 100.0%
test  accuracy 94.0%
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\long_curves.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\long_confusion.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\results\long_worst.png
wrote C:\Users\isamg\Documents\Mas1004 Files\Assignment 1\mas1004-assignment1\results\long_model.pt

One line for your experiment table:
| long | ImageNet | all layers | 20 | 0.0003 | 11,178,051 | 100.0% | 94.0% |

To put this model on your web page: python src/export_web.py --tag long
```

Problem 4. Cleaning

My rule

An image stays only if it is an ordinary photograph of the outsole of the right shoe, where the tread pattern is visible and its shape can be told apart. Each category must show: Converse, a sole with the diamond shape; Nike Air Force, a sole with clean lines and circular shapes; Vans, a sole with the honeycomb shape. I remove an image if it (1) shows the full shoe and the sole is out of frame, tiny or blurred; (2) shows something other than the outsole of a shoe (drawing, logo, text, collage, screenshot); (3) shows a sole that does not match the brand and the specific model of its category; (4) shows anything else that is not a shoe; (5) shows a sole that is destroyed, covered or mixed with other objects so that no pattern or shape can be seen; (6) is a near copy of an image I already keep. I keep angled soles as long as part of the pattern is visible and its shape can be told apart, worn or dirty soles unless no shape or pattern can be seen at all, and two soles in one picture if both are the correct brand and model. The full rule is in `outputs/rule.txt`. I wrote it before removing anything.

Three kinds of junk







Three kinds of junk, one picture of each (chosen by Claude from the removed images of my first collection round): (a) whole shoe, sole not visible (junk_no_tread.jpg, a silver Converse high-top seen from the side, the rule-1 junk that was the biggest group, 186 of the first 246 removals); (b) not a photo (junk_not_a_photo.jpg, a labelled drawing of a Nike Air Force 1 with the sole shown as an illustration); (c) wrong object (junk_wrong_object.jpg, a van with a honeycomb pattern painted on

it, which is where the pattern comes from something that is not a shoe).

What I removed

```
class                    downloaded  removed   kept  other
converse_diamond_sole_shape        700      619     81      0
nike_air_force_circular_sole_shape        452      358     94      0
vans_honeycomb_sole_shape        427      334     93      0

reasons
  1291  not-kept
    20  duplicate

"other" is images that are in data/raw but neither in data/clean nor removed with `clean.py
↪  remove`, for example ones you deleted by hand. Their reasons are not recorded.
```

I chose to describe the rule instead of estimating a reason for each image. The removal table shows 1,291 removed as "not-kept" (I chose the images to keep by my rule, so a reason for each removed image was not recorded) and 20 as "duplicate". The report explains the rule instead.

Near copies

`clean.py suspects` listed 181 pairs of near copies (similarity above 0.95). Joined together, they form 96 groups covering 240 images; the biggest group has 11 images. That run was made on the 1,168 images that existed before I added 411 Converse images. In my keep lists I then found 20 more duplicates, which I removed (4 Converse, 11 Nike, 5 Vans) so that only one picture of each stayed. The pairs from the suspects run are drawn here:



The suspects

I deleted almost all except for one. There were a lot of images deleted after cleaning as many downloaded were not right. from 400 it went down to just a bit less than 100 because most of it was junk, very easy to confuse with the whole shoe, or many images with watermarks. Numbers from the files: the first suspects run listed the 20 most suspicious images in each class, and all 20 in each class (Converse, Nike, Vans) ended up removed (I kept Converse image 0091. It is not among those 20: the 20 Converse suspects were 0068, 0079, 0114, 0144, 0040, 0127, 0229, 0122, 0024, 0029, 0011, 0104, 0010, 0233, 0211, 0142, 0126, 0249, 0118 and 0065. Image 0091 ranked 23rd, just outside the top 20, and the model gave it only a 14.9% chance of being Converse, so it is a suspect that I kept because it shows the Converse pattern). That run was made before the extra 411 Converse images were added, so for Converse it covers the first 289 images. It also listed 181 near-copy pairs over all images. In my keep lists I found 20 duplicates, which were removed (4 Converse, 11 Nike, 5 Vans).

Before and after cleaning

```
MAS1004 Assignment 1, checking your work
```

```
Training images  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\clean)
------------------------------------------------------------------------------------------------------
↪  ---
  converse_diamond_sole_shape     81 images
  nike_air_force_circular_sole_shape    94 images
  vans_honeycomb_sole_shape       93 images

Your images from a new source  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\new_images)
------------------------------------------------------------------------------------------------------
↪  ----------------------
  converse_diamond_sole_shape     23 images
  nike_air_force_circular_sole_shape    16 images
  vans_honeycomb_sole_shape        5 images

The same image appearing twice
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

The model you exported for the web page
---------------------------------------
  ok       3 classes: converse_diamond_sole_shape, nike_air_force_circular_sole_shape,
  ↪  vans_honeycomb_sole_shape
  ok       model.onnx is 44.8 MB, 11,178,051 parameters
           input [1, 3, 224, 224], prepared as: shorter side 256, centre 224 x 224, mean [0.485,
           ↪  0.456, 0.406], std [0.229, 0.224, 0.225]

Does the exported model give the answers Python gave?
-----------------------------------------------------
  ok       yes, they agree (largest gap 0.0000)

Your images from a new source  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\new_images)
------------------------------------------------------------------------------------------------------
↪  ----------------------

  accuracy on your new images: 42/44 = 95.5%

  rows are what it really is, columns are what the model said
                                       converse nike_air vans_hon
    converse_diamond_sole_shape              21        2        0
    nike_air_force_circular_sole_shape        0       16        0
    vans_honeycomb_sole_shape                 0        0        5

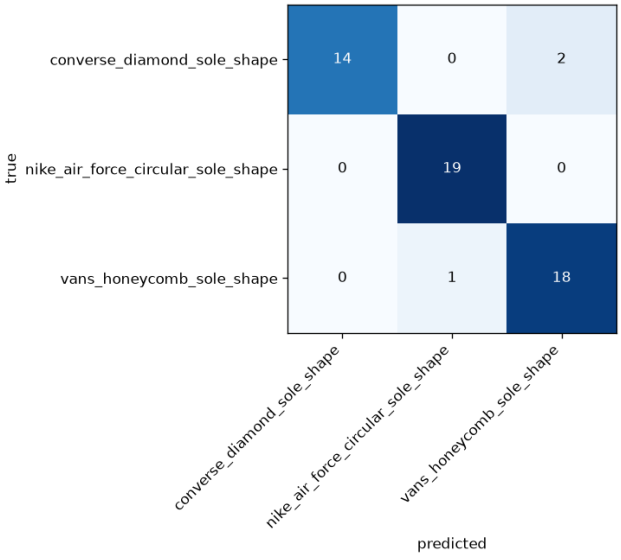
  the mistakes it was most sure about:
    IMG-20261013-WA0008.jpg      said nike_air_force_circular_sole_shape (58%), really
    ↪  converse_diamond_sole_shape
    IMG-20261013-WA0016.jpg      said nike_air_force_circular_sole_shape (49%), really
    ↪  converse_diamond_sole_shape

Your saved runs on your new images  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\new_images)
------------------------------------------------------------------------------------------------------
↪  --------------------------
  run               test accuracy   new images
  run                       93.3%        93.2%   (41/44)
  clean                     94.4%        95.5%   (42/44)

Summary
-------
  nothing is broken
  1 thing(s) worth a sentence in your report:
    - the biggest class has 23 images and the smallest has 5. A model can score well just by always
    ↪  guessing the big class.
```

	images trained on	own test set	my 44 new images
run (before cleaning)	1,264	93.3% (315 images)	93.2% (41/44)
clean (after cleaning)	214	94.4% (54 images)	95.5% (42/44)

Nothing went down. Test accuracy went from 93.3% (run) to 94.4% (clean), and on my 44 new photos from 93.2% (41/44) to 95.5% (42/44). The two test sets are not the same images, and the cleaned model trained on 268 images instead of 1,579, so the 1.1-point difference on the test set is small and could be chance (the cleaned test set is only 54 images). The comparison on the same 44 new photos is the fairer one, and it is 1 photo better.



The cleaned model's confusion matrix on its own 54 test images: Converse 14 of 16 (2 called Vans), Nike 19 of 19, Vans 18 of 19 (1 called Nike), 3 mistakes in all.

Problem 5. Images from a new source

The new images came from my own shoes, my friends' shoes, and my family's shoes. I asked them to send me pictures if they had the same model of shoe and sole.

```
describing 44 images with ResNet18 ...
none of your 44 new images is a near copy of any of the 1579 images you downloaded

the closest any new image comes to a downloaded one: 0.89 (near copies are above 0.95)
```

The new images are 23 Converse, 16 Nike and 5 Vans, 44 in all (`outputs/check.txt`).

Downloaded test set against new images

model	downloaded test set	new images
run (before cleaning)	93.3%	93.2% (41/44)
clean (after cleaning)	94.4%	95.5% (42/44)

On the new images the cleaned model gets all 16 Nike and all 5 Vans right and 21 of 23 Converse (2 called Nike). The checker warns that the classes have 23, 16 and 5 images, so a model could score well by always guessing the big class.

The confident mistakes



IMG-20261013-WA0008: said Nike (58%), really Converse



IMG-20261013-WA0016: said Nike (49%), really Converse

There were only two mistakes on the 44 new photos, not five. Both were Converse photos that the model called Nike: IMG-20261013-WA0008.jpg (58% sure) and IMG-20261013-WA0016.jpg (49% sure). In both the shoe lies on its side, so the sole is seen at a steep angle and the pattern is squeezed. Both show a thick white wavy midsole with long curved lines, which resembles the clean lines and curves of the Nike category, and the diamond pattern is only a small part of the picture. Both also have large wood and furniture backgrounds. These are guesses from looking at the pictures, not tested.

What is different about my new images

They are a bit lower in quality, as they were taken without the intent of posting, so the light is not the best, it is very natural and not set up. Angles are diverse as we have the flexibility of taking pictures in different angles. Hands are visible in many pictures as the person is holding them to take the picture.

Problem 6. The demo

Address: `https://isabel-mg.github.io/mas1004-assignment1/` and the code is at `https://github.com/Isabel-mg/mas1004-assignment1`. The model runs inside the page, so there is no server and no photo leaves the machine. The badge is green: the self-test on the page says the browser agrees with Python (largest gap 0.0000), and `docs/model.onnx` is 44.8 MB with 11,178,051 parameters. The page shows the model of the run tagged `clean`.

The page has no "none of these" answer. It always shows the probabilities of my three classes.

It was not able to be very certain compared to actual shoe soles. I tried with a shirt and a skirt. It was not able to have more than 60% on a class. I also tried with other soles of different shoes, those in comparison went up to 65% or so but never exceeded 70% unless the shape was very similar. The one class that did get confused with one of the other soles I tried with was Vans, but I purposely chose a very similar sole.

For six other images that I downloaded (the first three are soles of another brand, the last three are patterns), the model gave these answers (printed by code, `outputs/problem7_answers.txt`):



Problem 7. What the model looks at

I made 11 kinds of change to my own new photos. I asked Claude to make the copies, and I chose every change and every guess myself. Six photos (two per category) were used for ten of the changes, and ten photos (two Converse, four Nike, four Vans) for the shuffle. Under each picture the answer of the exported model and its confidence are printed, green if the answer is right and red if it is wrong; the same numbers are in the table in the appendix. The copies are in `data/changed/`.

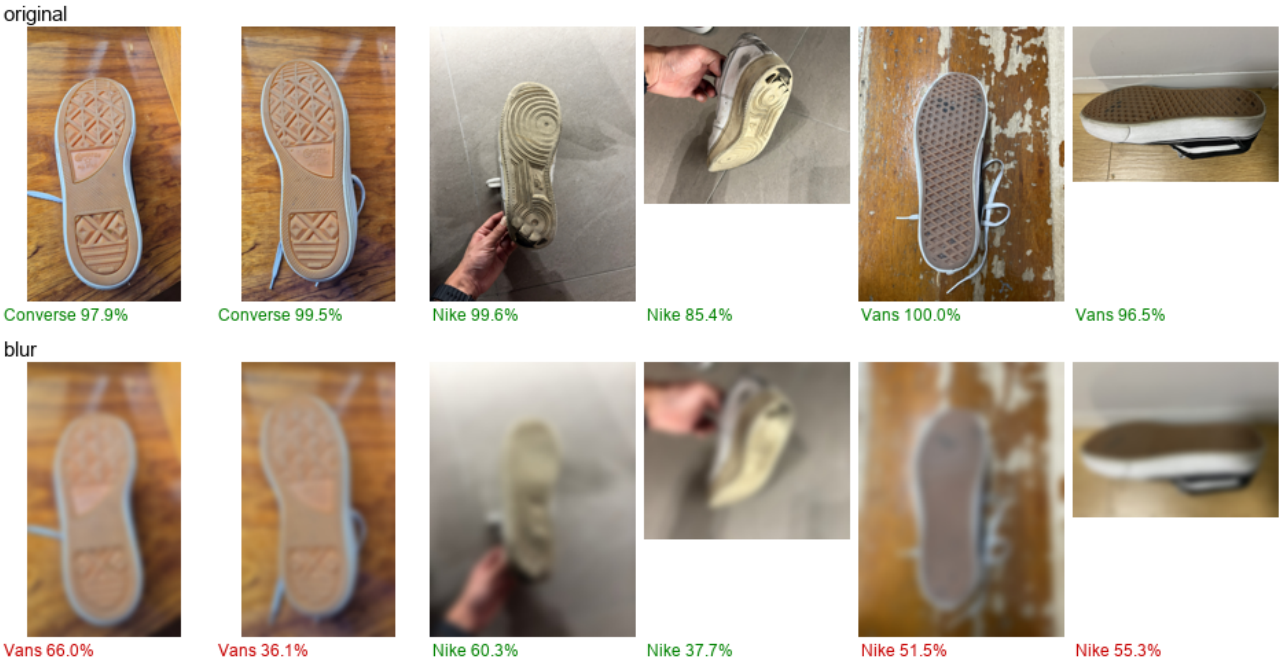
My requests to the agent, word for word:

1. can you make a new folder with the images i took and have them modified? first change the color for some, then for orders and a grid, for others extreme zoom in, and for others something else that might help me find the guess. choose a few images and have the change for all so i can try the same image but with the different changes. for example image 001 has 4 versions, the gray, the grid, the zoom, etc. i also downloaded these images from other soles to see how it works.
2. 1. just a grid, like when you take pictures and you have a 3x3 grid, 2. one can be blurred, and have another one with only the back of the sole and other with only the front of the sole, the middle is not visible.
3. can you make more changed version with a smaller grid? so more of the image is covered with it and with a lor of small squares.
4. i have 3 more changes i want to try and test: negative or inverted contrast and maybe the model relies on specific way shadows fall, not color but shadows and highlights. scale, i want to see if it can recognize it even with different ratio and proportions, can we stretch the image by 50%. and the last one i want to test shape vs texture and see if it relies on the texture or the flat shapes, so can you convert it into edge-detected images so the background and surface textures are completely white/black, leaving only the literal linework of the diamond, circle, or honeycomb patterns.
5. 1 estra change trial: Cut the image into a 4x4 grid of squares, shuffle the positions of the squares randomly, and stitch them back together so the sole layout makes no sense, but the local textures are perfectly intact.
6. for this random trial can you add 2 more nike pictures and 2 more vans pictures?

Guess 1: Blur

Guess and why. For the blurred images I don't think it will work at all, the grid and shapes of the sole will not be visible and i dont think just color will work, except maybe with the nike, as they are different color.

What I did and what I saw. For the blur tests, they all failed except for the Nike pictures, although one has very low confidence. I believe it's because the shape is not clearly visible, and the model does not rely much on color and probably bases its guesses on shape.



Check against the code output. 4 of the 6 blurred photos were answered wrongly (both Converse photos as Vans, both Vans photos as Nike). Both Nike photos were still answered Nike, one with only 37.7%.

Guess 2: Grayscale

Guess and why. I will try with the grayscale pictures to confirm that color is not a factor for the guesses after checking blurred didn't work.

What I did and what I saw. Grayscale worked just fine. The weakest guess was Vans with 83.4%. With this I can see it does not rely on color.



Check against the code output. All 6 grayscale photos were answered correctly, and the weakest was Vans at 83.4%.

Guess 3: The 3x3 grid

Guess and why. I will try with the grid, i think it will confuse the model and the guesses might not be correct and we are changing the shape of the sole.

What I did and what I saw. The grid was not enough to confuse the model, but it did confuse one Nike shoe with Converse. I later tried a much smaller grid, instead of the 3x3. With this I am thinking maybe straight lines confuse the Nike guesses.



Check against the code output. 5 of 6 photos were answered correctly (92.1% to 99.8%). One Nike photo (1791805866921) was answered Converse with 52.7%, with Nike at 43.8%.

Guess 4: Extreme zoom

Guess and why. I will try with extreme zoom, i think the model relies on recognizing the pattern on various areas, instead of one very specific one. i expect it to still guess the right sole but not with much confidence.

What I did and what I saw. Confidence was high in most of the pictures, meaning that even with zoom Converse and Vans were guessed correctly, so it does not need a global view. For Nike, as long as certain curved lines are still visible, it can also be guessed.



Check against the code output. All 6 photos were answered correctly, with confidences from 78.0% to 99.5%.

Guess 5: Only a small part (first or last third)

Guess and why. Similar to the zoom in, I will try with just a very small part of it. Now I expect good guesses even with the small part uncovered.

What I did and what I saw. With the gray box covering, depending on what was left visible, it determined the confidence and classification. For one of the Nike images that it did not guess, it was mainly because there were no curves visible. For the others, the clearer the pattern, the better the confidence. The wrong answer appeared only when the visible part could be mistaken for another pattern because of the light, the part shown or the angle.



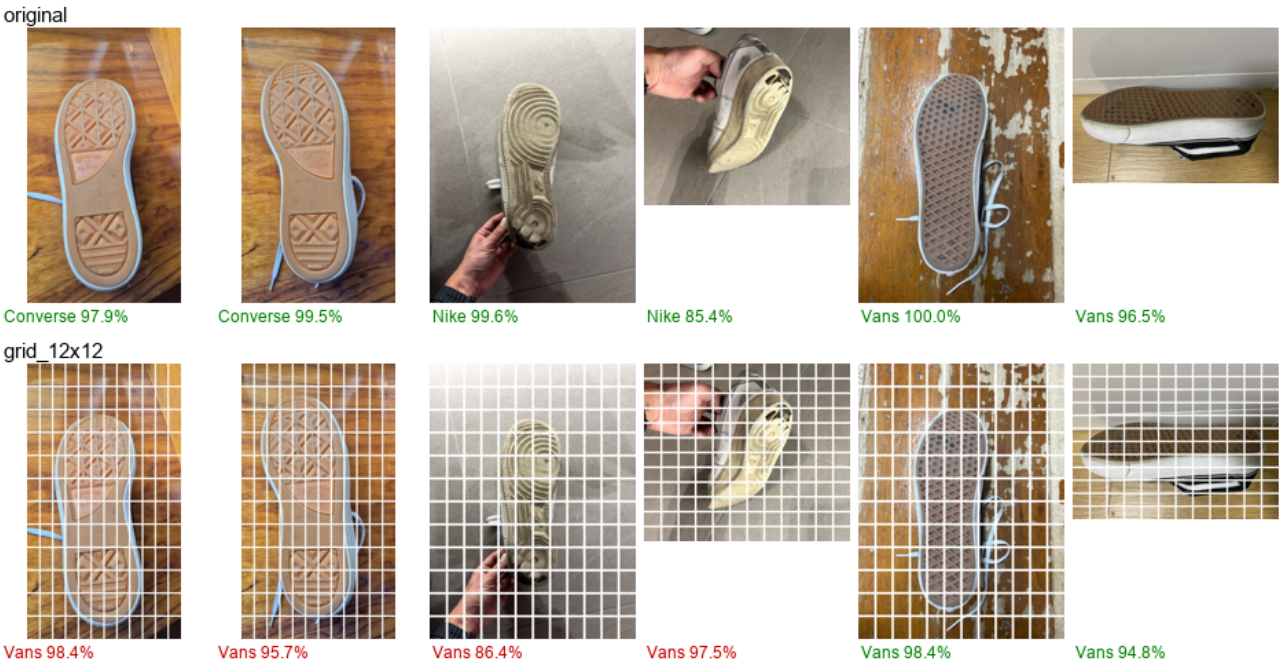


Check against the code output. Of the 12 answers, 11 were right. One Nike photo was answered Converse (43.0%) when only its last third was visible. Confidences ranged from 36.8% to 98.5%.

Guess 6: The finer 12x12 grid

Guess and why. With the larger grid, my guess is that it will get confused and not guess the right pattern and go for a different sole or very low confidence.

What I did and what I saw. All results said it was the Vans honeycomb. This makes sense as the grid might look similar to the shape of the honeycomb.



Check against the code output. All 6 photos were answered Vans, with 86.4% to 98.4%.

Guess 7: Inverted colors

Guess and why. I will try with inverted colors, I think it will still get it right and accurate.

What I did and what I saw. As suspected, it didn't change the result much, except for one where the light changes a lot and the sole was less clear and a bit blurry.



Check against the code output. All 6 inverted photos were answered correctly. The lowest was a Vans photo at 57.1%, against 96.5% for its original.

Guess 8: Stretched by 50%

Guess and why. Same for the stretch, I think it will get it right, just trying to discard other options.

What I did and what I saw. Yes, results were good and guesses high.



Check against the code output. All 6 stretched photos were answered correctly, with 74.0% to 99.8%.

Guess 9: Edges only

Guess and why. It will guess with the edges, and I think that is what it mainly uses.

What I did and what I saw. I think I am pretty sure it uses this, as the images with less clear edges had lower confidence and even one where the edges were very confusing. It got the wrong guess.



Check against the code output. 5 of 6 photos were answered correctly (70.3% to 99.3%). One Nike photo (1791805866921) was answered Converse with 56.3%.

Guess 10: Other images with similar patterns

Guess and why. I will try images with similar patterns and textures to see if it will get it right and confirm the edges hypothesis.

What I did and what I saw. It got Converse and Vans correct, especially Vans. For Nike, even though I found a similar shape, it did not guess it right.



Check against the code output. The dartboard and the tiles were answered Converse (68.4% and 62.1%), the fence Vans (92.7%). The three soles of another brand were answered Nike (55.6%), Vans (99.3%) and Nike (59.5%). None of the pattern images was answered Nike.

Guess 11: Shuffled 4x4 tiles

Guess and why. I will do one more trial to check what happens with Nike shoes. I will shuffle a 4x4 grid to see if Nike also relies on the same things.

What I did and what I saw. Again, Converse and Vans were guessed mostly right and with high confidence. One Vans and all Nike pictures were wrong. With this I can come to the conclusion that Vans and Converse rely on local texture or the edges of the geometric shape. Also, when I tried one of the New Balance shoes with a similar sole, it guessed Vans, so that is another confirmation. That

is also the reason why they can also get confused between each other sometimes. Nike relies on global structure, the circles and their position. If the sole is changed, it is easier to get confused.



Check against the code output. The two Converse photos were answered Converse (87.1%, 86.2%). Three of the four Vans photos were answered Vans (96.3%, 86.1%, 83.6%) and one Vans photo (1791811068662) was answered Converse (50.7%). All four Nike photos were answered Converse (56.4% to 93.7%).

What I now think the model uses

I am around 90% sure it does not rely on colors, lighting, or overall shoe context. Instead it relies on local texture detectors for Vans and Converse, looking for specific edges and line intersections to identify either diamond-shaped for Converse or hexagonal for Vans. Because it looks at local patches, it can identify even when the image is shuffled or zoomed in, but it gets confused with everyday objects with similar texture. For Nike, it relies on global geometry. It searches for continuous structure, which is the reason why it passed trial 10 but not trial 11. It is also the reason why it passed the zoom in but not the shuffled one: it is looking for continuous curved lines and continuous geometry, which the zoom in keeps intact, unlike the shuffling.

Every conclusion rests on six photos per change (ten for the shuffle), so it is an indication, not a proof. The 10 original photos were all answered correctly by the model before any change (80.7% to 100.0%).

Appendix. Output of the checks

check.py

```
MAS1004 Assignment 1, checking your work

Training images  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\clean)
------------------------------------------------------------------------------------------------┘
↪  ---
  converse_diamond_sole_shape     81 images
  nike_air_force_circular_sole_shape    94 images
  vans_honeycomb_sole_shape       93 images

Your images from a new source  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\new_images)
------------------------------------------------------------------------------------------------┘
↪  ----------------------
  converse_diamond_sole_shape     23 images
  nike_air_force_circular_sole_shape    16 images
  vans_honeycomb_sole_shape        5 images

The same image appearing twice
```

```
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

The model you exported for the web page
---------------------------------------
  ok       3 classes: converse_diamond_sole_shape, nike_air_force_circular_sole_shape,
  ↪  vans_honeycomb_sole_shape
  ok       model.onnx is 44.8 MB, 11,178,051 parameters
           input [1, 3, 224, 224], prepared as: shorter side 256, centre 224 x 224, mean [0.485,
           ↪  0.456, 0.406], std [0.229, 0.224, 0.225]

Does the exported model give the answers Python gave?
-----------------------------------------------------
  ok       yes, they agree (largest gap 0.0000)

Your images from a new source  (C:\Users\isamg\Documents\Mas1004 Files\Assignment
↪  1\mas1004-assignment1\data\new_images)
-------------------------------------------------------------------------------------------------------┘
↪  ----------------------

  accuracy on your new images: 42/44 = 95.5%

  rows are what it really is, columns are what the model said
                                       converse nike_air vans_hon
    converse_diamond_sole_shape              21        2        0
    nike_air_force_circular_sole_shape        0       16        0
    vans_honeycomb_sole_shape                 0        0        5

  the mistakes it was most sure about:
    IMG-20261013-WA0008.jpg      said nike_air_force_circular_sole_shape (58%), really
    ↪  converse_diamond_sole_shape
    IMG-20261013-WA0016.jpg      said nike_air_force_circular_sole_shape (49%), really
    ↪  converse_diamond_sole_shape

Summary
-------
  nothing is broken
  2 thing(s) worth a sentence in your report:
    - the biggest class has 23 images and the smallest has 5. A model can score well just by always
    ↪  guessing the big class.
    - docs/model.onnx has changed since your last commit. The published page still shows the old
    ↪  model.
```

Model answers for every original and changed image

change	true class	image	answer	converse	nike	vans
other_soles	-	1.jpg	nike	41.7%	55.6%	2.7%
other_soles	-	2.webp	vans	0.2%	0.5%	99.3%
other_soles	-	3.webp	nike	29.1%	59.5%	11.4%
similar_patterns	-	4.jpg	converse	68.4%	2.9%	28.7%
similar_patterns	-	5.jpg	converse	62.1%	2.1%	35.9%
similar_patterns	-	6.jpg	vans	3.8%	3.4%	92.7%
original	converse	IMG-20261013-WA0000.jpg	converse	97.9%	0.0%	2.1%
blur	converse	IMG-20261013-WA0000.jpg	vans	20.4%	13.6%	66.0%
edges	converse	IMG-20261013-WA0000.jpg	converse	99.2%	0.2%	0.6%
first_third_only	converse	IMG-20261013-WA0000.jpg	converse	56.6%	12.5%	30.9%
gray	converse	IMG-20261013-WA0000.jpg	converse	99.9%	0.0%	0.1%
grid_12x12	converse	IMG-20261013-WA0000.jpg	vans	1.1%	0.4%	98.4%
grid_3x3	converse	IMG-20261013-WA0000.jpg	converse	98.6%	0.0%	1.4%
invert	converse	IMG-20261013-WA0000.jpg	converse	99.4%	0.1%	0.6%
last_third_only	converse	IMG-20261013-WA0000.jpg	converse	68.5%	12.6%	18.9%
shuffle_4x4	converse	IMG-20261013-WA0000.jpg	converse	87.1%	2.9%	10.0%
stretch_150	converse	IMG-20261013-WA0000.jpg	converse	99.6%	0.0%	0.4%
zoom_extreme	converse	IMG-20261013-WA0000.jpg	converse	78.0%	3.1%	18.8%

original	converse	IMG-20261013-WA0001.jpg	converse	99.5%	0.0%	0.5%
blur	converse	IMG-20261013-WA0001.jpg	vans	30.2%	33.7%	36.1%
edges	converse	IMG-20261013-WA0001.jpg	converse	99.3%	0.2%	0.5%
first_third_only	converse	IMG-20261013-WA0001.jpg	converse	71.5%	12.2%	16.3%
gray	converse	IMG-20261013-WA0001.jpg	converse	100.0%	0.0%	0.0%
grid_12x12	converse	IMG-20261013-WA0001.jpg	vans	2.7%	1.6%	95.7%
grid_3x3	converse	IMG-20261013-WA0001.jpg	converse	99.4%	0.0%	0.6%
invert	converse	IMG-20261013-WA0001.jpg	converse	99.8%	0.1%	0.2%
last_third_only	converse	IMG-20261013-WA0001.jpg	converse	54.2%	19.3%	26.5%
shuffle_4x4	converse	IMG-20261013-WA0001.jpg	converse	86.2%	1.9%	11.9%
stretch_150	converse	IMG-20261013-WA0001.jpg	converse	99.8%	0.0%	0.2%
zoom_extreme	converse	IMG-20261013-WA0001.jpg	converse	80.7%	4.2%	15.1%
original	nike	1791805848253.jpg	nike	0.3%	99.6%	0.1%
blur	nike	1791805848253.jpg	nike	12.5%	60.3%	27.2%
edges	nike	1791805848253.jpg	nike	1.5%	97.9%	0.6%
first_third_only	nike	1791805848253.jpg	nike	5.0%	77.0%	18.0%
gray	nike	1791805848253.jpg	nike	0.4%	99.4%	0.2%
grid_12x12	nike	1791805848253.jpg	vans	5.3%	8.3%	86.4%
grid_3x3	nike	1791805848253.jpg	nike	7.2%	92.1%	0.7%
invert	nike	1791805848253.jpg	nike	0.2%	99.8%	0.0%
last_third_only	nike	1791805848253.jpg	nike	23.8%	71.1%	5.2%
shuffle_4x4	nike	1791805848253.jpg	converse	61.2%	27.5%	11.3%
stretch_150	nike	1791805848253.jpg	nike	0.7%	99.2%	0.2%
zoom_extreme	nike	1791805848253.jpg	nike	15.0%	79.7%	5.3%
original	nike	1791805866921.jpg	nike	10.9%	85.4%	3.6%
blur	nike	1791805866921.jpg	nike	30.5%	37.7%	31.8%
edges	nike	1791805866921.jpg	converse	56.3%	21.0%	22.7%
first_third_only	nike	1791805866921.jpg	nike	21.0%	57.7%	21.3%
gray	nike	1791805866921.jpg	nike	7.3%	90.5%	2.2%
grid_12x12	nike	1791805866921.jpg	vans	0.8%	1.7%	97.5%
grid_3x3	nike	1791805866921.jpg	converse	52.7%	43.8%	3.5%
invert	nike	1791805866921.jpg	nike	2.3%	97.2%	0.6%
last_third_only	nike	1791805866921.jpg	converse	43.0%	25.4%	31.5%
shuffle_4x4	nike	1791805866921.jpg	converse	93.7%	0.9%	5.4%
stretch_150	nike	1791805866921.jpg	nike	17.6%	74.0%	8.4%
zoom_extreme	nike	1791805866921.jpg	nike	9.3%	81.4%	9.3%
original	nike	1791805871496.jpg	nike	14.4%	80.7%	4.8%
shuffle_4x4	nike	1791805871496.jpg	converse	91.1%	5.2%	3.7%
original	nike	20261013_003516.jpg	nike	0.0%	99.9%	0.1%
shuffle_4x4	nike	20261013_003516.jpg	converse	56.4%	22.3%	21.3%
original	vans	1791808843445.jpg	vans	0.0%	0.0%	100.0%
blur	vans	1791808843445.jpg	nike	13.2%	51.5%	35.3%
edges	vans	1791808843445.jpg	vans	11.1%	1.8%	87.2%
first_third_only	vans	1791808843445.jpg	vans	0.6%	0.9%	98.5%
gray	vans	1791808843445.jpg	vans	0.1%	0.0%	99.9%
grid_12x12	vans	1791808843445.jpg	vans	0.5%	1.1%	98.4%
grid_3x3	vans	1791808843445.jpg	vans	0.2%	0.0%	99.8%
invert	vans	1791808843445.jpg	vans	0.0%	0.0%	99.9%
last_third_only	vans	1791808843445.jpg	vans	41.7%	12.7%	45.6%
shuffle_4x4	vans	1791808843445.jpg	vans	2.8%	0.9%	96.3%
stretch_150	vans	1791808843445.jpg	vans	0.2%	0.1%	99.7%
zoom_extreme	vans	1791808843445.jpg	vans	1.0%	0.0%	98.9%
original	vans	1791811068662.jpg	vans	1.5%	1.9%	96.5%
blur	vans	1791811068662.jpg	nike	18.1%	55.3%	26.6%
edges	vans	1791811068662.jpg	vans	14.9%	14.8%	70.3%
first_third_only	vans	1791811068662.jpg	vans	16.0%	10.8%	73.2%
gray	vans	1791811068662.jpg	vans	8.8%	7.7%	83.4%
grid_12x12	vans	1791811068662.jpg	vans	2.4%	2.8%	94.8%
grid_3x3	vans	1791811068662.jpg	vans	1.2%	0.7%	98.1%
invert	vans	1791811068662.jpg	vans	28.9%	14.1%	57.1%
last_third_only	vans	1791811068662.jpg	vans	35.6%	27.6%	36.8%
shuffle_4x4	vans	1791811068662.jpg	converse	50.7%	5.0%	44.3%
stretch_150	vans	1791811068662.jpg	vans	2.3%	4.8%	92.9%
zoom_extreme	vans	1791811068662.jpg	vans	0.5%	0.1%	99.5%
original	vans	1791811068730.jpg	vans	0.2%	0.3%	99.5%
shuffle_4x4	vans	1791811068730.jpg	vans	9.7%	4.2%	86.1%
original	vans	1791811068749.jpg	vans	0.1%	0.1%	99.9%

```
shuffle_4x4        vans       1791811068749.jpg           vans        13.7%    2.7%   83.6%
```

Cleaning rule

```
MY CLEANING RULE (second round, written before running clean.py look/suspects on the new 1,168
↪  images)
================================================================================

An image stays only if it is an ordinary photograph of the OUTSOLE (the
bottom) of the right shoe, where the tread pattern is visible and its shape can
be told apart. I remove everything else.

WHAT EACH CATEGORY MUST SHOW
  converse_diamond_sole_shape          Converse sole with the diamond shape
  nike_air_force_circular_sole_shape   Nike Air Force sole with clean lines and
                                       circular shapes
  vans_honeycomb_sole_shape            Vans sole with the honeycomb shape

I REMOVE an image if it:
  1. shows the full shoe and the sole is out of frame, tiny or blurred
  2. shows something other than the outsole of a shoe (drawing, logo, text,
     collage, listing screenshot, placeholder, patent/CAD schematic)
  3. shows a sole that does not match the brand and the specific model sole
     of its category
  4. shows anything else that is not a shoe (tiles, flooring, cars, insoles,
     a booklet alone)
  5. shows a sole that is destroyed, covered or mixed with other objects, so
     that no pattern or shape can be seen
  6. is a near copy of an image I already keep

I KEEP (borderline cases I decided):
  - Angled soles, as long as part of the pattern is visible and its shape can
    be told apart.
  - Worn or dirty soles, unless no shape or pattern can be seen at all.
  - Two soles in one picture, if both are the correct brand and model.

REASON WORDS for `clean.py remove <class>/<file> ... --reason "<word>"`
(copied verbatim into data/removed/log.csv, so `clean.py count` reads straight
off this rule):
  no-tread       rule 1, and rule 5 when the sole is covered or too worn or
                 dirty to see any pattern
  not-a-photo    rule 2
  wrong-object   rule 3 (wrong brand, model or pattern) and rule 4 (not a shoe)
  duplicate      rule 6
  unreadable     a file the tools cannot open

BUDGET: check.py wants more than 50 images per class after cleaning, and the
biggest class must not exceed 3x the smallest. Target: at least 100 kept per
class. Report the final counts either way.
```